

VÍCTOR ÁLVAREZ FERNÁNDEZ

Unidad:

Unidad 4 - Automatización de Tareas

Práctica:

P41-Automatizacion-Tareas

Fecha:

18 de Febrero de 2026

1. Herramientas para la Automatización
2. Rutinas: Funciones y Procedimientos
 - *Funciones*
 - *Procedimientos*
 - *Llamadas*
 - *Sentencias: SELECT y SELECT INTO*
 - *Otras Operaciones y Consultas*
 - *Parámetros y Variables*
 - *Sentencias: Condicionales*
 - *Sentencias: Bucles*
 - *Cursores*
 - *Manejo de Excepciones*
3. Disparadores
4. Eventos: Planificación de Tareas

MySQL cuenta con dos posibles herramientas para la automatización de tareas, la primera de ellas cuenta con entorno gráfico y se llama MySQL Enterprise Monitor; la otra alternativa y la más recomendada porque implica tener un amplio conocimiento es basada en la programación a través de Procedimientos Almacenados.

Los Procedimientos Almacenados son secuencias de código que se almacenan del lado del Servidor y que aportan una serie de ventajas:

- Permiten incluir de Instrucciones Compuestas (bucles, condicionales, ...)
- Reducen la sobrecarga de la red al estar almacenadas en el Servidor
- Permiten incluir cálculos complejos
- Permiten estandarizar operaciones de cálculo
- Proporcionan un mecanismo para procesar errores
- Cuentan con un control de acceso de datos sensibles mediante privilegios otorgados

Métodos de Ejecución

- **Funciones:** devuelven el resultado de un cálculo y se usan en expresiones
- **Procedimientos:** se utilizan para realizar cálculos generales o generar conjuntos de resultados que se muestran al cliente
- **Disparadores:** se asocian a una tabla y se ejecutan cuando esta se modifica con los eventos INSERT, UPDATE y DELETE
- **Eventos:** se ejecutan en un momento concreto según su programación

Conjunto de Comandos almacenados en el Servidor, que son llamados por los clientes.

- **Funciones:** devuelven un sólo valor
- **Procedimientos:** pueden devolver varios valores o incluso ninguno

Funciones

Sintaxis: **CREATE DEFINER** = 'usuario'@'host' **FUNCTION** nomFuncion() **SQL SECURITY** {DEFINER | INVOKER} **BEGIN** sentencias; **END**

Procedimientos

Sintaxis: **CREATE DEFINER** = 'usuario'@'host' **PROCEDURE** nomProcedimiento() **SQL SECURITY** {DEFINER | INVOKER} **BEGIN** sentencias; **END**

Nivel de Seguridad

- **DEFINER:** se ejecuta con los permisos del creador de la rutina
- **INVOKER:** se ejecuta con los permisos del cliente que lo invoca

Se recomienda que el usuario creador sea el administrador de la SGBD y que cuente con los permisos de Consultas (SELECT) y de Ejecución (EXECUTE) en todas las bases de datos y tablas del Servidor para mayor comodidad a la hora de crear las rutinas.

Si cuando se crea la rutina no se indica la cláusula **SQL SECURITY**, por defecto el valor de la misma será **DEFINER**.

Si en la rutina se utiliza el valor **INVOKER**, será necesario que el usuario que haga la llamada desde el lado del cliente cuente con permisos de Ejecución (EXECUTE) para poder ejecutarla.

Uso de Delimitadores: *DELIMITER*

En MySQL, el carácter punto y coma (;) se utiliza por defecto para indicar el final de una sentencia. Sin embargo, al crear rutinas que contienen múltiples instrucciones en su interior, surge un conflicto: el servidor interpretaría el primer punto y coma interno como el final de toda la creación, dejando el código incompleto y generando un error de sintaxis. Para solucionar esto, se utiliza la instrucción **DELIMITER \$\$**. Esto le indica a MySQL que cambie temporalmente el símbolo de finalización por **\$\$**, permitiendo que todo el bloque de código se procese como una única unidad. Una vez definida la rutina, se emplea **DELIMITER ;** para restablecer el punto y coma como el delimitador estándar para las consultas habituales.

```
3      -- Permisos de Lectura y Ejecución sobre tabla mysql.user --
4 •    GRANT SELECT, EXECUTE ON *.* TO pruebadevuelgo;
5
6      -- Rol por Defecto para Usuario --
7 •    SET DEFAULT ROLE ALL TO pruebadevuelgo;
8 •    FLUSH PRIVILEGES;
9
10     -- Procedimiento contar usuarios --
11     DELIMITER $$
12 •    CREATE DEFINER = 'pruebadevuelgo'@'%'
13     PROCEDURE contador_usuarios()
14     BEGIN
15         SELECT 'Número de Cuentas: ', COUNT(*)
16         FROM mysql.user;
17     END $$
18     DELIMITER ;
19
20 •    -- Llamada al Procedimiento --
21     CALL contador_usuarios();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Número de Cuentas:	COUNT(*)
▶	Número de Cuentas:	9

Llamadas

- Funciones: **SELECT nomFuncion()**;
- Procedimientos: **CALL nomProcedimiento()**;

Sentencias: *SELECT* y *SELECT INTO*

SELECT

- **Funciones:** NO muestran el resultado en pantalla
- **Procedimientos:** Si muestran el resultado en pantalla

SELECT INTO

- **Funciones:** almacena el resultado en una variable
- **Procedimientos:** almacena el resultado en una variable

```
24  -- PROCEDIMIENTO: PRODUCTO DE GAMA ALTA --
25  DELIMITER $$
26  • CREATE PROCEDURE gestionar_producto(IN id_buscado INT)
27  BEGIN
28      DECLARE precio_actual DECIMAL(8,2);
29      -- SELECT: Muestra el resultado directamente en la pantalla --
30      SELECT nombre, precio FROM productos WHERE id_producto = id_buscado;
31
32      -- SELECT ... INTO: Guarda el precio en una variable precio_actual --
33      SELECT precio INTO precio_actual FROM productos WHERE id_producto = id_buscado;
34
35      -- Uso variable precio_actual --
36      IF precio_actual > 500 THEN
37          SELECT 'Este es un producto de gama alta';
38      END IF;
39  END $$
40  DELIMITER ;
41
42  • CALL gestionar_producto(1);
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	Este es un producto de gama alta			
▶	Este es un producto de gama alta			

Otras Operaciones y Consultas

Eliminar una Rutina

Sintaxis: **DROP {FUNCTION | PROCEDURE} [IF EXISTS] {nomFuncion | nomProcedimiento};**

```
44      -- Eliminar Procedimiento --
45 •    DROP PROCEDURE IF EXISTS gestionar_producto;
```

Consultar Rutinas Creada

Sintaxis: **SHOW CREATE {FUNCTION | PROCEDURE} {nomFuncion | nomProcedimiento};**

```
47      -- Consultar Procedimiento Creado --
48 •    SHOW CREATE PROCEDURE gestionar_producto;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
Procedure	sql_mode	Create Procedure			
▶ gestionar_producto	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,...	CREATE DEFINER='root'@'localhost' PROCEDURE `gestionar_producto`(IN id_buscado INT) ...			

Consultar Estado de Todos los Procedimientos

Sintaxis: **SHOW PROCEDURE STATUS;**

```
50      -- Consultar Estado TODOS los procedimientos --
51 •    SHOW PROCEDURE STATUS;
```

Result Grid					Filter Rows:	Export:	Wrap Cell Content:
	Db	Name	Type	Definer			
▶	asxbd	contador_usuarios	PROCEDURE	pruebadefuego@%			
	asxbd	gestionar_producto	PROCEDURE	root@localhost			
	sys	create_synonym_db	PROCEDURE	mysql.sys@localhost			

Consultar Estado de un Procedimiento según un patrón

Sintaxis: **SHOW PROCEDURE STATUS LIKE** *'g%'*;

```
53 -- Consultar Estado de los procedimientos según un 'patrón' --
54 • SHOW PROCEDURE STATUS LIKE 'g%';
```

Result Grid						
Filter Rows:		Export:		Wrap Cell Content:		
	Db	Name	Type	Definer	Modified	Created
▶	asxbd	gestionar_producto	PROCEDURE	root@localhost	2026-02-11 19:37:08	2026-02-

Parámetros y Variables

Declaración de Variables: DECLARE

Crea una variable con el nombre y el tipo de dato indicado. Los datos pueden ser de tipo valor (INT, FLOAT, DECIMAL,...) de tipo fecha (DATE, TIME, YEAR,...) o de tipo cadena (CHAR, VARCHAR, TEXT, ...).

Sintaxis: **DECLARE** *nomVariable* *tipo_dato*;

Una variable se puede declarar con un valor por defecto (inicial o de arranque) que puede tener un valor concreto o uno nulo:

- I. Valor Concreto: **DECLARE** *a* *INT* **DEFAULT** 5;
- II. Valor NULL: **DECLARE** *a* *INT* **DEFAULT**;

Declaración de Variables: SET

Crea una variable utilizando el operador =.

Sintaxis: **SET** *nomVariable* = *valorNumerico*;

Parámetros

Permiten pasar datos a las rutinas, e incluso devolverlos en el caso concreto de los procedimientos. Estos son los tipos de parámetros que se pueden utilizar:

- **IN:** Parámetros de Entrada (por defecto)
- **OUT:** Parámetros de Salida (sólo en los Procedimientos). El procedimiento puede asignar valores a estos parámetros que son devueltos cuando se realiza una llamada.
- **INOUT:** Parámetros de Entrada/Salida (sólo en los Procedimientos). Permiten pasar valores, que podrán ser modificados.

Alcance de las Variables: no se puede ver el valor de una variable fuera del procedimiento salvo que el **parámetro sea de tipo OUT** o la **variable sea de tipo sesión (@)**.

```
56  -- EJEMPLO PROCEDIMIENTO CON PARÁMETRO IN (ENTRADA) --
57  DELIMITER $$
58  • CREATE PROCEDURE consultar_precio_producto(IN nombre_prod VARCHAR(80))
59  ○ BEGIN
60      -- El valor entra por 'nombre_prod' y se usa en el WHERE
61      SELECT nombre, precio
62      FROM productos
63      WHERE nombre = nombre_prod;
64  END $$
65  DELIMITER ;
66
67  • -- LLAMADA:
68  CALL consultar_precio_producto('Televisor 55"');
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	nombre	precio
▶	Televisor 55"	599.99

Rutinas: Funciones y Procedimientos

```
70  -- EJEMPLO PROCEDIMIENTO CON PARÁMETRO ON (SALIDA) --
71  DELIMITER $$
72  • CREATE PROCEDURE contar_productos_categoria(IN id_cat INT, OUT total INT)
73  BEGIN
74      -- Cuenta los productos de una categoría y lo guarda en 'total'
75      SELECT COUNT(*) INTO total
76      FROM productos
77      WHERE id_categoria = id_cat;
78  END $$
79  DELIMITER ;
80
81  • -- LLAMADA:
82  CALL contar_productos_categoria(1, @cuantos);
83  • SELECT @cuantos;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	@cuantos			
▶	2			

```
85  -- PROCEDIMIENTO CON PARÁMETROS ENTRADA/SALIDA (INOUT) --
86  DELIMITER $$
87  • CREATE PROCEDURE aplicar_descuento_acumulado(INOUT mi_valor DECIMAL(10,2))
88  BEGIN
89      DECLARE descuento DECIMAL(10,2) DEFAULT 10.00;
90      -- Resta el descuento al valor que entró y actualiza la misma variable
91      SET mi_valor = mi_valor - descuento;
92  END $$
93  DELIMITER ;
94
95  • -- LLAMADA:
96  SET @precio_variable = 750.00;
97  • CALL aplicar_descuento_acumulado(@precio_variable);
98
99  • SELECT @precio_variable;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	@precio_variable			
▶	740.00			

Condicionales

Condicional Simple: IF

Tiene solamente una condición.

```
271  -- CONDICIONAL SIMPLE (UNA CONDICIÓN) --
272  DELIMITER $$
273  • CREATE PROCEDURE comprobar_oferta(IN id_prod INT)
274  BEGIN
275      DECLARE precio_prod DECIMAL(8,2);
276
277      SELECT precio INTO precio_prod
278      FROM productos
279      WHERE id_producto = id_prod;
280
281      -- Estructura condicional simple --
282      IF precio_prod < 20.00 THEN
283          SELECT 'Este producto entra en la categoría: GANGAS' AS Etiqueta;
284      END IF;
285
286  END $$
287  DELIMITER ;
288
289  • -- Comprobar producto con id_prod = 5 --
290  CALL comprobar_oferta(5);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Etiqueta
▶	Este producto entra en la categoría: GANGAS

Condicional Múltiple: IF - ELSEIF - ELSE

Puede tener varias condiciones. No se recomienda su uso con más de tres condiciones.

```
295 • CREATE PROCEDURE clasificar_producto(IN id_prod INT)
296 BEGIN
297     DECLARE precio_prod DECIMAL(8,2);
298     DECLARE categoria_precio VARCHAR(20);
299
300     SELECT precio INTO precio_prod
301     FROM productos
302     WHERE id_producto = id_prod;
303
304     -- Condicional
305     IF precio_prod < 20.00 THEN
306         SET categoria_precio = 'GANGA';
307     ELSEIF precio_prod BETWEEN 20.00 AND 100.00 THEN
308         SET categoria_precio = 'ESTÁNDAR';
309     ELSE
310         SET categoria_precio = 'PREMIUM';
311     END IF;
312
313     SELECT nombre, precio, categoria_precio AS 'Clasificación'
314     FROM productos
315     WHERE id_producto = id_prod;
316
317 END $$
318 DELIMITER ;
319
320 • CALL clasificar_producto(6);
```

Result Grid			
Filter Rows:			
Export:  Wrap Cell Content: 			
	nombre	precio	Clasificación
▶	Coche teledirigido	29.90	ESTÁNDAR

Condicional Múltiple: CASE

Puede tener varias condiciones. Se recomienda su uso a partir de una cuarta condición.

```
322  -- CONDICIÓN MÚLTIPLE: CASE --
323  DELIMITER $$
324  • CREATE PROCEDURE analizar_pedido(IN id_p INT)
325  BEGIN
326      DECLARE valor_total DECIMAL(10,2);
327
328      SELECT total INTO valor_total
329      FROM pedidos
330      WHERE id_pedido = id_p;
331
332      -- CASE --
333      CASE
334          WHEN valor_total < 50.00 THEN
335              SELECT 'Prioridad: BAJA' AS Logística, 'Empaquetado estándar' AS Notas;
336          WHEN valor_total <= 200.00 THEN
337              SELECT 'Prioridad: MEDIA' AS Logística, 'Revisar stock antes de enviar' AS Notas;
338          WHEN valor_total <= 600.00 THEN
339              SELECT 'Prioridad: ALTA' AS Logística, 'Requiere firma de entrega' AS Notas;
340          ELSE
341              SELECT 'Prioridad: CRÍTICA' AS Logística, 'Envío asegurado y urgente' AS Notas;
342      END CASE;
343  END $$
344  DELIMITER ;
345
346  • CALL analizar_pedido(3);
```

Result Grid		Filter Rows:	Export: 	Wrap Cell Content: 
	Logística	Notas		
►	Prioridad: CRÍTICA	Envío asegurado y urgente		

Bucles

Bucle Simple: LOOP

El bucle se repite hasta que se ejecute la sentencia **LEAVE**.

```
182  -- BUCLE LOOP --
183  DELIMITER $$
184  • CREATE PROCEDURE contador_basico(INOUT n INT)
185  BEGIN
186      -- Comienza el Bucle LOOP --
187      ciclo: LOOP
188          -- El bucle finaliza cuando n=10 --
189          IF n = 10 THEN
190              LEAVE ciclo;
191              SELECT 'Se acabó';
192          END IF;
193          -- Contador (Se incrementa en 1 en cada pasada) --
194          SET n = n + 1;
195      END LOOP ciclo;
196  END $$
197  DELIMITER ;
198
199  • SET @numero = 3;
200  • CALL contador_basico(@numero);
201  • SELECT @numero;
```

Result Grid

	@numero
▶	10

Bucle con Condición: REPEAT - UNTIL

El bucle se repite hasta que se cumpla una condición.

```
203 -- BUCLE: REPEAT UNTIL --
204 DELIMITER $$
205 • CREATE PROCEDURE ejemplo_repeat(INOUT contador INT)
206   BEGIN
207     -- Comienza el Bucle REPEAT --
208     REPEAT
209       SET contador = contador + 1;
210     UNTIL contador > 3
211     END REPEAT;
212
213     SELECT 'Terminó el contador';
214   END $$
215 DELIMITER ;
216
217 • SET @numero = 1;
218 • CALL ejemplo_repeat(@numero);
219 • SELECT @numero;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	@numero			
▶	4			

Bucle con Condición: WHILE

El bucle se repite mientras se cumple una condición.

```
222  -- BUCLE: WHILE --
223  DELIMITER $$
224  • CREATE PROCEDURE ejemplo_while(INOUT contador INT)
225  BEGIN
226      -- Comienza el Bucle WHILE --
227      WHILE contador < 4 DO
228          SET contador = contador + 1;
229      END WHILE;
230
231      SELECT 'Terminó el contador';
232  END $$
233  DELIMITER ;
234
235  • SET @numero = 1;
236  • CALL ejemplo_while(@numero);
237  • SELECT @numero;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	@numero			
▶	4			

Cursores

Los cursores son herramientas que permiten recorrer una tabla, fila por fila para extraer un resultado con una consulta (SELECT).

Sintaxis: **DECLARE** *nomCursor* CURSOR FOR *sentencia_SELECT*;

La declaración de los cursores se debe realizar tras la declaración de todas las variables que llevará la rutina.

Manipulación de Cursores

- **OPEN**: Inicializa el conjunto de resultados asociados al cursor. **OPEN** *nomCursor*;
- **FETCH**: Extrae contenido de las filas (una o varias columnas). **OPEN** *nomCursor* INTO *variable (columna)*;
- **CLOSE**: Cierra el cursor. **CLOSE** *nomCursor*;

```
239  -- CURSORES --
240  DELIMITER $$
241  • CREATE PROCEDURE cursor_minimo()
242  BEGIN
243      DECLARE id_actual INT;
244      DECLARE fin INT DEFAULT 0;
245
246      -- Declaración del Cursor --
247      DECLARE mi_cursor CURSOR FOR
248      SELECT id_producto FROM productos;
249
250      -- Controlador de Errores: --
251      -- La variable 'fin' puede tener cualquier valor distinto de 0 --
252      DECLARE CONTINUE HANDLER FOR NOT FOUND
253      SET fin = 1;
254
```

```
255 OPEN mi_cursor; -- Apertura Cursor
256 bucle: LOOP
257     FETCH mi_cursor INTO id_actual; -- Extrae columna
258     IF fin = 1 THEN
259         LEAVE bucle;
260     END IF;
261     SELECT id_actual;
262 END LOOP;
263 CLOSE mi_cursor; -- Cierre Cursor
264 END $$
265 DELIMITER ;
266
267 • -- Muestra Número de Filas de la Tabla productos --
268 CALL cursor_minimo();
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	id_actual			
▶	8			

Manejo de Excepciones

Las excepciones permiten manejar situaciones NO deseadas que pueden producirse durante la ejecución de una rutina. Para ello, se utiliza la instrucción **HANDLER**. Esta instrucción se encarga de indicar a MySQL que hacer en caso de que se produzca un error.

Sintaxis: **DECLARE** *tipo_actuacion* **HANDLER** FOR *condición* acciones;

Tipos de Actuación

- **CONTINUE**: se ejecuta el "plan B" y el procedimiento continua su curso
- **EXIT**: se ejecuta el "plan B" y el procedimiento finaliza inmediatamente

Condición

- **Código de Error de MySQL**: código de MySQL que indica el tipo de error (Ej: 1062)
- **SQLSTATE 'código'**: códigos de estado de MySQL que tienen un significado concreto. También, podemos encontrar dentro de esta gama los **SQLException** y **SQLWarning**, siendo ambos muy habituales
- **NOT FOUND**: es la condición utilizada para indicar que el cursor llega al final y se desea volver al inicio

Ejemplo Excepción con CONTINUE

```
271      -- MANEJO DE ERRORES: CONTINUE --
272      DELIMITER $$
273 •    CREATE PROCEDURE ejemplo_continue()
274      BEGIN
275          -- Si ocurre CUALQUIER error (SQLEXCEPTION), imprime un mensaje y continúa --
276          DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
277              SELECT 'Algo salió mal, pero continuamos...';
278
279          -- Forzamos un error: Intentamos insertar un nombre NULO (suponiendo que no puede serlo)
280          -- O simplemente una tabla que no existe
281          INSERT INTO tabla_fantasma (nombre) VALUES ('Error');
282
283          -- Gracias al CONTINUE, esta línea SÍ se ejecutará
284          SELECT 'He llegado al final del procedimiento a pesar del error' AS Resultado;
285      END $$
286      DELIMITER ;
287
288 •    CALL ejemplo_continue;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	Resultado			
▶	He llegado al final del procedimiento a pesar del error			

Ejemplo Excepción con EXIT

```
291  -- MANEJO ERRORES: EXIT --
292  DELIMITER $$
293  • CREATE PROCEDURE ejemplo_exit()
294  ○ BEGIN
295      -- Si hay un duplicado (Código 1062), avisa y finaliza el procedimiento --
296      DECLARE EXIT HANDLER FOR 1062
297          SELECT 'ERROR FATAL: El ID ya existe. Operación cancelada.' AS Alerta;
298
299      -- Primera inserción
300      INSERT INTO productos (id_producto, nombre, precio) VALUES (101, 'Ratón', 15.00);
301
302      -- Segunda inserción: Fuerzo el error usando el mismo ID
303      INSERT INTO productos (id_producto, nombre, precio) VALUES (101, 'Teclado', 25.00);
304  END $$
305  DELIMITER ;
306
307  • CALL ejemplo_exit();
```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	Alerta
▶	ERROR FATAL: El ID ya existe. Operación cancelada.

Tipo especial de rutina almacenada que se activa cuando ocurre un evento de tipo INSERT, UPDATE o DELETE. Los disparadores (triggers) se almacenan en la Base de Datos en la que se definen.

Momento en el que actúa un Disparador

- **BEFORE** (antes): es ideal para corregir datos antes de guardarlos
- **AFTER** (después): es muy útil para actualizar otras tablas o dejar un registro de incidencias

Eventos de un Disparador

- **INSERT**: se utiliza para insertar datos en una tabla. Cuando se utiliza en un disparador permite el uso del calificador **NEW** para referirse a los nuevos datos a insertar en una columna.
- **UPDATE**: se utiliza para actualizar datos en una tabla. Cuando se utiliza en un disparador permiten el uso de los calificadores **NEW** para referirse a los posibles nuevos datos, y **OLD** para referirse a los datos actuales de la tabla.
- **DELETE**: se utiliza para eliminar datos de una tabla. Cuando se utiliza en un disparador permite el uso del calificador **OLD** para referirse a los datos actuales de la tabla.

Sintaxis: **CREATE TRIGGER** *nomDisparador* {**BEFORE** | **AFTER**} {**INSERT** | **UPDATE** | **DELETE**} **ON** *nomTabla* **FOR EACH ROW** *sentencia*;

Ejemplo de Disparador AFTER - UPDATE

```
311 -- DISPARADOR ACTUALIZAR STOCK DE PRODUCTOS --
312 DELIMITER $$
313 • CREATE TRIGGER actualizar_stock_pedido
314 AFTER INSERT ON detalle_pedido -- El evento ocurre en la Tabla 'detalle_pedido'
315 FOR EACH ROW
316 BEGIN
317     UPDATE stock -- El disparador actualiza la tabla 'stock'
318         -- La columna 'unidades' pertenece a la tabla 'stock'
319         -- La columna 'cantidad' pertenece a la tabla 'detalle_pedido'
320     SET unidades = unidades - NEW.cantidad
321     -- id_producto es la clave en común entre ambas tablas
322     WHERE id_producto = NEW.id_producto;
323 END $$
324 DELIMITER ;
325
326 • -- Comprobar Unidades (Tabla 'stock') del producto con id = 1 (Televisor) --
327 SELECT id_producto, unidades FROM stock WHERE id_producto = 1;
328
```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	id_producto	unidades
▶	1	45

```
329 -- Simular una venta (Tabla 'detalle_pedido') --
330 • INSERT INTO detalle_pedido (id_pedido, id_producto, cantidad, subtotal)
331 VALUES (1, 1, 5, 2999.95);
332
333 -- Comprobar nuevamente Unidades (Tabla 'stock') --
334 • SELECT id_producto, unidades FROM stock WHERE id_producto = 1;
```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	id_producto	unidades
▶	1	40

Ejemplo de Disparador BEFORE - INSERT

```
337 -- DISPARADOR PARA CONTROLAR PUNTUACIÓN DE VALORACIÓN --
338 DELIMITER $$
339 • CREATE TRIGGER validar_puntuacion
340 BEFORE INSERT ON valoraciones -- El evento ocurre en tabla 'valoraciones'
341 FOR EACH ROW
342 BEGIN
343     IF NEW.puntuacion > 5 THEN
344         SET NEW.puntuacion = 5;
345     ELSEIF NEW.puntuacion < 1 THEN
346         SET NEW.puntuacion = 1;
347     END IF;
348 END $$
349 DELIMITER ;
350
351 • -- Simular una puntuación nueva --
352 INSERT INTO valoraciones (id_cliente, id_producto, puntuacion, comentario, fecha)
353 VALUES (1, 2, 10, 'Increíble', 20260217);
354
355 -- Comprobamos puntuaciones igual 5 (es la máxima posible) --
356 • SELECT id_cliente, puntuacion, comentario FROM valoraciones WHERE puntuacion = 5;
```

Result Grid					Filter Rows:	Export:	Wrap Cell Content:
	id_cliente	puntuacion	comentario				
▶	1	5	Muy buena calidad				
	4	5	Excelente portátil				
	1	5	Increible				

Ejemplo de Disparador BEFORE - UPDATE (OLD - NEW)

```
359      -- DISPARADOR ACTUALIZACIÓN PRECIOS --
360      DELIMITER $$
361      CREATE TRIGGER protector_precios
362      BEFORE UPDATE ON productos
363      FOR EACH ROW
364      BEGIN
365          -- Comparamos el valor NUEVO con el VIEJO
366          -- Si el precio nuevo es menor a la mitad del precio viejo
367          IF NEW.precio < (OLD.precio / 2) THEN
368              -- Cancelamos la rebaja excesiva y mantenemos el precio existente
369              SET NEW.precio = OLD.precio;
370          END IF;
371      END $$
372      DELIMITER ;
```

Ejemplo de Disparador AFTER - DELETE

```
375 -- DISPARADOR: ELIMINAR COMENTARIO BORRADO --
376 DELIMITER $$
377 • CREATE TRIGGER eliminar_comentario_borrado
378 AFTER DELETE ON valoraciones
379 FOR EACH ROW
380 BEGIN
381     SET @comentario_eliminado = OLD.comentario;
382 END $$
383 DELIMITER ;
384
385 • -- Comprobar Valoraciones Actuales:
386 SELECT id_valoracion, comentario FROM valoraciones;
387
388 -- Eliminar Primer Comentario:
389 • DELETE FROM valoraciones WHERE id_valoracion = 1;
390
391 -- Comprobar en variable de sesión el comentario eliminado:
392 • SELECT @comentario_eliminado;
```

Result Grid |  |  Filter Rows: | Edit:    | Export/Import: 

	id_valoracion	comentario
▶	1	Muy buena calidad
	2	Funciona bien
	3	Aceptable
	4	Excelente portátil

387

Result Grid |  |  Filter Rows: | Export:  | Wrap Cell Content: 

	@comentario_eliminado
▶	Muy buena calidad

Consultas Disparadores

Sintaxis: **SHOW TRIGGERS;**

```
394      -- CONSULTAR DISPARADORES BASE DE DATOS ACTUAL --
395 •    SHOW TRIGGERS;
```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 					
	Trigger	Event	Table	Statement	Timing
▶	actualizar_stock_pedido	INSERT	detalle_pedido	BEGIN UPDATE stock -- El disparador actualiz...	AFTER
	protector_precios	UPDATE	productos	BEGIN -- Comparamos el valor NUEVO con el ...	BEFORE
	aviso_borrado	DELETE	productos	BEGIN -- Guardamos el nombre del producto ...	AFTER
	validar_puntuacion	INSERT	valoraciones	BEGIN IF NEW.puntuacion > 5 THEN SE...	BEFORE

Eliminar Disparador

Sintaxis: **DROP TRIGGER nomDisparador;**

```
397      -- ELIMINAR UN DISPARADOR --
398 •    DROP TRIGGER eliminar_comentario_borrado;
```

Los eventos son tareas programadas que se ejecutan en un momento dado.

Comprobar Encendido del Planificador

Sintaxis: **SHOW PROCESSLIST;**

```
403 -- Comprobar Encendido Planificador --
404 • SHOW PROCESSLIST;
```

Result Grid								
Filter Rows:		Export:		Wrap Cell Content:				
	Id	User	Host	db	Command	Time	State	Info
▶	5	event_scheduler	localhost	NULL	Daemon	14311	Waiting on empty queue	NULL

Activar / Desactivar el Planificador

El planificador se puede activar o desactivar de la misma manera que se activa una variable global de tipo booleana.

- Encendido: **SET GLOBAL event_scheduler = ON;**
- Apagado: **SET GLOBAL event_scheduler = OFF;**

Crear un Evento de Planificación

Sintaxis: **CREATE EVENT nomEvento ON SCHEDULE {AT | EVERY} DO sentencias;**

Tipos de Planificación

- **AT:** se ejecuta en una fecha y hora concreta
- **EVERY:** se ejecuta periódicamente (cada hora, día, semana, mes, ...)

Evento con Ejecución Concreta: AT

```
407 -- Evento Rebajas --
408 • CREATE EVENT rebajas
409 ON SCHEDULE AT '2026-02-18 10:00:00'
410 DO
411 UPDATE productos
412 SET precio = precio * 0.90
413 WHERE id_categoria = 1;
```

Evento con Ejecución Periódica: *EVERY*

```
416 -- Ejemplo Añadir 5 unidades mensuales a todos los productos --
417 DELIMITER //
418 • CREATE EVENT reposicion_mensual_stock
419 ON SCHEDULE EVERY 1 MONTH
420 DO
421 BEGIN
422     UPDATE stock
423     SET unidades = unidades + 5;
424 END //
425 DELIMITER ;
```

Ver todos los Eventos

Sintaxis: **SHOW EVENTS;**

```
427 • -- VER EVENTOS --
428 SHOW EVENTS;
```

Result Grid			
		Filter Rows:	Export
	Db	Name	Definer
▶	asxbd	rebajas	root@localhost
	asxbd	reposicion_mensual_stock	root@localhost

Modificar un Evento

Sintaxis: **ALTER EVENT *nomEvento* ON SCHEDULE EVERY 2 MONTH;**

```
430 -- Modificar un Evento --
431 • ALTER EVENT reposicion_mensual_stock
432 ON SCHEDULE EVERY 2 MONTH;
```

Eliminar un Evento

Sintaxis: **DROP EVENT *nomEvento*;**

```
434 -- Eliminar Evento --
435 • DROP EVENT rebajas;
```

Claúsulas

Cuando se crea un evento se pueden utilizar una serie de cláusulas que nos permitan establecer una fecha y/o hora de inicio (**STARTS**), una fecha y/o hora de fin (**ENDS**), e incluso una cláusula para almacenar el evento aunque este haya expirado (**ON COMPLETION PRESERVE**)

```
438      -- EVENTO SEMANA LIQUIDACIÓN --
439      DELIMITER //
440 • CREATE EVENT liquidacion
441      ON SCHEDULE EVERY 1 HOUR
442      STARTS '2026-02-18 00:00:00' -- Empieza el día 18
443      ENDS '2026-02-22 23:59:59'  -- Termina el día 22
444      ON COMPLETION PRESERVE -- No se borra
445      DO
446      BEGIN
447          UPDATE productos
448          SET precio = precio * 0.80
449          WHERE id_categoria = 2; -- Categoría Hogar
450      END //
451      DELIMITER ;
```

Desactivar o Activar Eventos (Tareas)

En un momento dado, se puede optar por desactivar o activar un evento (tarea) por una determinada situación.

Sintaxis Desactivar: **ALTER EVENT nomEvento DISABLE;**

Sintaxis Activar: **ALTER EVENT nomEvento ENABLE;**

```
453 • -- DESACTIVAR EVENTO --
454      ALTER EVENT liquidacion DISABLE;
455
456      -- ACTIVAR EVENTO --
457 • ALTER EVENT liquidacion ENABLE;
```